

ARI-LLM: Autoregressive Imputation for Network Traffic Matrix via Large Language Models

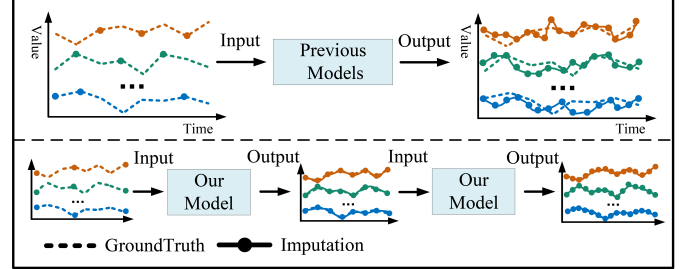
Abstract—The Traffic Matrix (TM) plays a vital role in a wide range of network management tasks, yet monitoring complete TMs is prohibitively expensive. Recent efforts focus on recovering incomplete TMs via learning-based models by learning from historical TM snapshots. However, existing approaches often suffer from poor generalization and require frequent retraining when the network environment evolves—let alone unseen networks. In this paper, we propose a novel TM imputation framework, named ARI-LLM, based on Large Language Models (LLMs). Inspired by human-like cognitive process of coarse-to-fine reasoning, our method incrementally refines the incomplete TM through a step-wise autoregressive mechanism, which is compatible with the pretraining paradigm of LLMs, endowing our model with powerful generalization and zero-shot capabilities. To ensure plug-and-play deployment across TM imputation tasks in dynamic environments, we craft a flow-aware tokenization scheme that supports scalable and topology-adaptive embeddings. Extensive experiments demonstrate that ARI-LLM significantly outperforms state-of-the-art methods on large-scale sparse TMs (over 90% missingness), reducing imputation error by up to 34.7%. It maintains strong performance whether given only limited TM knowledge (e.g., 20% observed flows, 5% training snapshots) or no prior knowledge at all on unseen networks. The reproducible experimental codes are available at <https://anonymous.4open.science/r/ARI-LLM-E7A3>.

Index Terms—Network measurement, traffic matrix imputation, large language models, autoregressive learning.

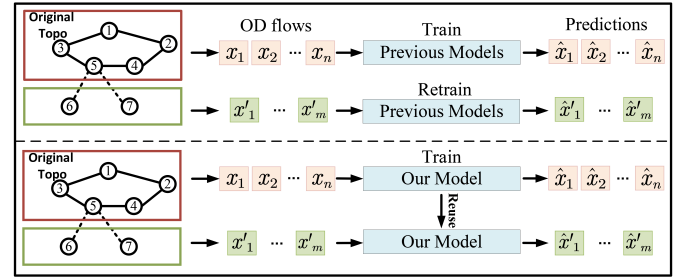
I. INTRODUCTION

A Traffic Matrix (TM) represents the set of traffic volumes exchanged between nodes in a network over a given time interval. Accurately monitoring the TM is crucial for a wide range of downstream network tasks, such as capacity planning [1], anomaly detection [2], and traffic engineering [3] [4].

TMs can be collected using diverse techniques, including SNMP [5] [6], NetFlow [7], OpenTM [8], or methods based on routing information [9] [10]. However, as network scales continue to expand, directly measuring all OD flows by capturing each packet trace incurs prohibitively high administrative and computational costs [11] [12]. Even worse, missing values in TMs are often inevitable in most real-world networks, with missing rates of over 90% being fairly common. Beyond cost-saving considerations, several practical constraints account for this phenomenon [13]. Firstly, many networks are undergoing a transition between legacy and modern devices, and not all devices are equipped with the capability to monitor and store the enormous flow-level traffic data. Secondly, for privacy-preserving reasons, monitoring end-to-end traffic is often prohibited in certain sensitive regions of the network. Lastly, even if all devices support flow-level monitoring and no privacy restrictions are imposed, the collected TM is often incomplete due to practical issues such as device failures or network congestion. Therefore, whether due to the high



(a) One-step imputation vs. autoregressive imputation



(b) Low generalization vs. high generalization

Fig. 1. Illustration of previous models vs. our proposed model. (a) Previous models apply one-step imputation to recover the entire TM in a single pass, whereas our autoregressive model iteratively predicts a finer TM based on a coarser one. (b) Previous models require retraining once new flows are involved, whereas our model adapts to new flows without retraining.

cost of comprehensive monitoring or unavoidable practical limitations, accurately recovering missing entries in the TM is of critical importance for enabling effective downstream network optimization tasks.

With the rapid advancements in neural network technologies, learning-based TM imputation methods [14] have surpassed traditional low-rank matrix factorization approaches [15] [16], becoming the new state-of-the-art. These methods first learn TM patterns from historical snapshots, and then recover the missing flows by inferring the most likely values that align with the learned pattern based on the observed flows [17]. However, these methods face several critical challenges in practical deployments: (1) *High uncertainty in one-step imputation*. Almost all existing methods adopt a one-step imputation strategy (as illustrated in the top part of Fig. 1(a)). Such learning paradigm tends to be unstable, particularly under high missing rates (e.g., over 90% missingness). With only one-step supervision, the model often fails to converge effectively, resulting in highly fluctuating outputs. (2) *Limited model generalizability*. Almost all imputation models are tailored to specific network configurations present in the training dataset. When the network topology changes—such as the addition or removal of nodes—it becomes necessary to

recollect training data, redesign the neural architecture, and retrain the model from scratch, incurring significant human and computational overhead. (3) *Dependence on rich historical TM data*. Many methods assume the availability of a long-term TM training data to learn the traffic pattern comprehensively. However, in practice, especially in dynamic or constrained networks, obtaining a long period of TM data is extremely difficult, making it challenging to train the model adequately.

Since the TM data in practical networks is often limited (either incomplete or with short term) and network structures often frequently evolve, a fundamental question raises: **Can we develop a TM imputation method that, once trained—even with limited TM measurements—can be plug-and-play across diverse network environments? Ideally, such a method should accurately recover highly sparse TMs, generalize well to network changes without retraining, and even perform zero-shot imputation on entirely unseen networks with unknown topologies and TM prior knowledge.**

The emerging large language models (LLMs), with their powerful pretrained knowledge and zero-shot reasoning capabilities, offer new opportunity for addressing the above challenges. Recently, the rapid advancement of LLMs has led to transformative breakthroughs across a wide range of domains [18] [19] [20]. Although a few recent LLM-based models have been developed for networking-related tasks [21] [22], most of them treat LLMs merely as typical neural networks, applying end-to-end training or fine-tuning strategies. In fact, the powerful comprehension ability of LLMs is largely attributed to their autoregressive training paradigm that mimics human learning processes [23].

In this paper, we take the first step toward designing an LLM-based model specifically tailored for the TM imputation tasks. Inspired by the autoregressive training paradigm of LLMs, our new model adopts a coarse-to-fine autoregressive imputation strategy (as illustrated in the bottom part of Fig. 1(a)), which closely aligns with human cognitive processes. Compared to one-step imputation, the autoregressive paradigm not only enables LLMs to quickly grasp the underlying essence of TM imputation task, but also enhances the stability of the model’s reasoning process.

Furthermore, to ensure high generalization capability to dynamic network changes or even unseen networks, we craft a tokenization scheme for the TM imputation. Specifically, different from conventional tokenization strategies that treat time steps as basic units, we instead perform tokenization at the level of individual flows. Additionally, we demonstrate both theoretically and empirically that including all flows in each training epoch does not always lead to expected performance; instead, there exists an optimal number of flows that maximizes the model’s effectiveness. This design offers two key advantages: First, flow-level tokenization allows the model to flexibly adapt to TM dimensional changes by simply adjusting the number of input tokens—just like LLMs handle sentences of varying lengths—without requiring any modifications to the model’s input layer (as illustrated in Fig. 1(b)). Second, flow-level tokenization allows the model to better exploit spatial correlations among flows. Compared to traditional time-series dependencies, capturing inter-flow correlations in networks is

more critical for accurate TM imputation.

In summary, this paper makes the following key contributions to the field of TM measurement:

- We propose an LLM-based method for TM imputation, which adopts a coarse-to-fine imputation strategy to mimic human-like reasoning and fully exploit the capabilities of LLMs. Specifically, we decompose the input TM data into a sequence of multi-resolution representations, and employ an autoregressive framework that enables the model to progressively predict a finer-resolution TM conditioned on the coarser one.
- We design a flow-aware tokenization scheme, task-specific prompt template, and lightweight fine-tuning strategy for TM imputation, enabling the LLMs to effectively align with the TM imputation tasks. These designs allow the model to handle dynamic network environments without requiring any structural modifications or retraining. They also endow the model with strong few-shot and zero-shot learning capabilities, making it possible to seamlessly transfer to unseen network environments.
- We conduct extensive experiments on three real-world TM datasets to compare ARI-LLM with the state-of-the-arts. The results demonstrate that ARI-LLM achieves high accuracy under extreme sparsity (90%+ missing), improving the accuracy of the best baseline by up to 34.7%; generalizes well to network evolutions (only 7.1% drop on accuracy with 80% newly-added flows) and unseen networks (only 8.2% drop on average); converges rapidly (within 2~3 training epochs).

II. RELATED WORK

In this section, we first review the mainstream TM imputation methods, including traditional approaches, and learning-based approaches. Then we review recent advances in networking-related tasks using LLMs, and highlight the key differences between our proposed method and these existing approaches.

A. Traditional TM Imputation

Traditional TM imputation methods include Network Tomography (NT)-based approaches and Matrix/Tensor Decomposition (M/TD)-based techniques.

NT-based methods estimate fine-grained OD flows using linear equations from link loads and the routing matrix. Since these equations are often rank-deficient, NT-based methods address the ill-posed problem by introducing assumptions about the TM distribution [24] [12] [25]. For instance, Vardi [24] and Tebaldi et al. [12] assumed a Poisson distribution for network traffic. Cao et al. [26] assume a Gaussian traffic model, while Zhang et al. [27] applied a gravity model. M/TD-based methods first extract the principal components of the TM through matrix or tensor decomposition, and then reconstruct the full TM entries based on these components [15] [16]. Matthew et al. [16] utilized K-Nearest Neighbors (KNN) to model the similarity patterns among different OD pairs and introduced a spatial constraint matrix to improve imputation accuracy. Paul et al. [15] leveraged the principle of maximum entropy to reconstruct missing entries in TMs. Xie et al. [28] exploited

the local structures of TMs to form lower-rank tensors from local correlations which significantly enhanced the imputation accuracy. Tian et al. [29] proposed a tensor-based imputation method that encodes continuous distance information into each tensor and integrates a sample selection mechanism to reduce the sampling rate. However, these methods perform well only under simple or idealized traffic distributions. They suffer from high computational costs and limited imputation performance when recovering intricate traffic flows.

B. Learning-Based TM Imputation

To better capture the complex correlations within TM, recent studies have adopted neural networks for TM imputation. Xie et al. [14] proposed NTC, which captures third-order interactions among traffic features via outer products. DATC proposed in [17] leverages a generative adversarial network (GAN) to recover the missing values in tensor-structured TMs. Autotomo proposed in [30] addressed the challenge of TM estimation under incomplete and biased training data. Ouyang et al. [31] proposed LTP, which boosts recovery accuracy by combining time embeddings with attention-enhanced LSTM. Liu et al. [32] introduces a distributed tensor completion method based on convolutional neural networks (CNNs), enhancing both scalability and accuracy. Wang et al. [13] proposed PetTC, a pairwise joint embedding-based tensor completion method that improves TM imputation by leveraging cross-domain embeddings and contrastive learning. Despite their superior performance in capturing the complex correlations within TMs, these methods still face notable limitations as mentioned above.

C. LLM-based Networking Approaches

LLMs have recently emerged as promising tools for general-purpose of networking applications. For example, Wu et al. [22] proposed NetLLM for diverse networking tasks, including viewport prediction, adaptive bitrate streaming, and cluster job scheduling. Cui et al. [21] introduced TrafficLLM, a dual-stage fine-tuning framework designed to learn generalizable traffic representations from heterogeneous traffic data, enabling both traffic classification and packet generation. Zhou et al. [33] proposed a GPT-4-based software agent framework that simulates knowledgeable researchers to assist in Internet incident investigations. Despite the remarkable breakthroughs achieved by these methods, they remain inadequate for solving the TM imputation problem.

In contrast to the aforementioned methods, we propose an LLM-based model specifically designed for TM imputation tasks. To the best of our knowledge, this is the first TM imputation approach with strong generalization capabilities: can adapt to network changes and even generalize to entirely unseen network environments, all without the need for model modification or retraining.

III. METHODOLOGY

In this section, we present our proposed model, named ARI-LLM, which employs LLMs for flexible and generalizable imputation for severely missing TMs.

A. Problem Formulation

We model the network as a graph $G = (V, E)$, where V denotes the set of nodes and E represents the set of links connecting them. Let $X_t = \{x_{t1}, x_{t2}, \dots, x_{tN}\}$, and $N = |V| \times |V|$, denotes the collection of traffic flows between all network nodes, at the t -th time point. The TM is typically formulated as a $T \times N$ matrix, $\mathbf{X} = \{X_1, X_2, \dots, X_T\}^\top$, represents the traffic flows measured during T time points. We use a sampling indication matrix $\mathbf{M} \in \mathbb{R}^{T \times N}$ to indicate the observed and missing values in $\mathbf{X}$. If x_{ti} is observed, $m_{ti} = 1$; otherwise, $m_{ti} = 0$. Let $\tilde{\mathbf{X}} = \mathbf{M} \odot \mathbf{X}$ denote the TM with only the observed values, where $\odot$ represents the element-wise product. In this paper, our goal is to recover all missing values in $\mathbf{X}$ given the observed values $\tilde{\mathbf{X}}$.

B. Model Overview

The overall framework of our proposed ARI-LLM is illustrated in Fig. 2. ARI-LLM performs progressive TM imputation in an autoregressive manner. For a training sample of TM $\mathbf{X}$, we first decompose it into sequences of different resolutions $\mathcal{X} = \{\mathbf{X}^{(1)}, \mathbf{X}^{(2)}, \dots, \mathbf{X}^{(h)}\}$. At the s -th autoregressive step, the model predicts the next resolution $\mathbf{X}^{(s+1)}$ based on the current resolution $\mathbf{X}^{(s)}$, until the high-fidelity TM $\hat{\mathbf{X}}$ is generated. In each autoregressive step, the input TM $\mathbf{X}^{(s)}$ is firstly processed by a crafted Flow2Vec Embedder, which maps each flow onto the token space of the LLM. Simultaneously, a Prompt Embedder encodes the designed prompt into tokens. The two embeddings are then concatenated and fed into the LLM body, whose output is subsequently passed through a MLP to map it onto a finer-grained TM.

We next provide a detailed explanation of the key components of our framework, including the autoregressive imputation mechanism, the Flow2Vec Embedder, the design of the Prompt Template, the LLM body, and the lightweight fine-tuning strategy.

C. Autoregressive Imputation Mechanism

Unlike existing methods that rely on one-step imputation, ARI-LLM adopts a novel autoregressive imputation mechanism that progressively predict the complete TM from coarse to fine granularity. On the one hand, under high missing rates, this multi-step imputation strategy significantly enhances the stability of the imputation process. On the other hand, this human-aligned, stepwise reasoning paradigm allows the LLMs to better capture the underlying traffic patterns, and grasp the essence of the TM imputation task.

To enforce LLMs to perform autoregressive imputation, two key problems must be addressed: First, how to determine the appropriate resolution for each imputation step; Second, how to effectively guide the model to generate the next, finer-grained TM based on the current resolution.

To address the first problem, we develop a coarse-to-fine strategy that doubles the resolution of the observations at each autoregressive step, inspired by human-like imputation process. Consider a TM with a total length of w time steps: $\mathbf{X} = \{X_1, X_2, \dots, X_w\}^\top$, we aim to construct a set of observation index collections $\mathcal{I} = \{\mathcal{I}^0, \mathcal{I}^1, \dots, \mathcal{I}^h\}$, where $\mathcal{I}^s$ ($s = \{0, 1, \dots, h\}$) denotes the set of observed time indices at

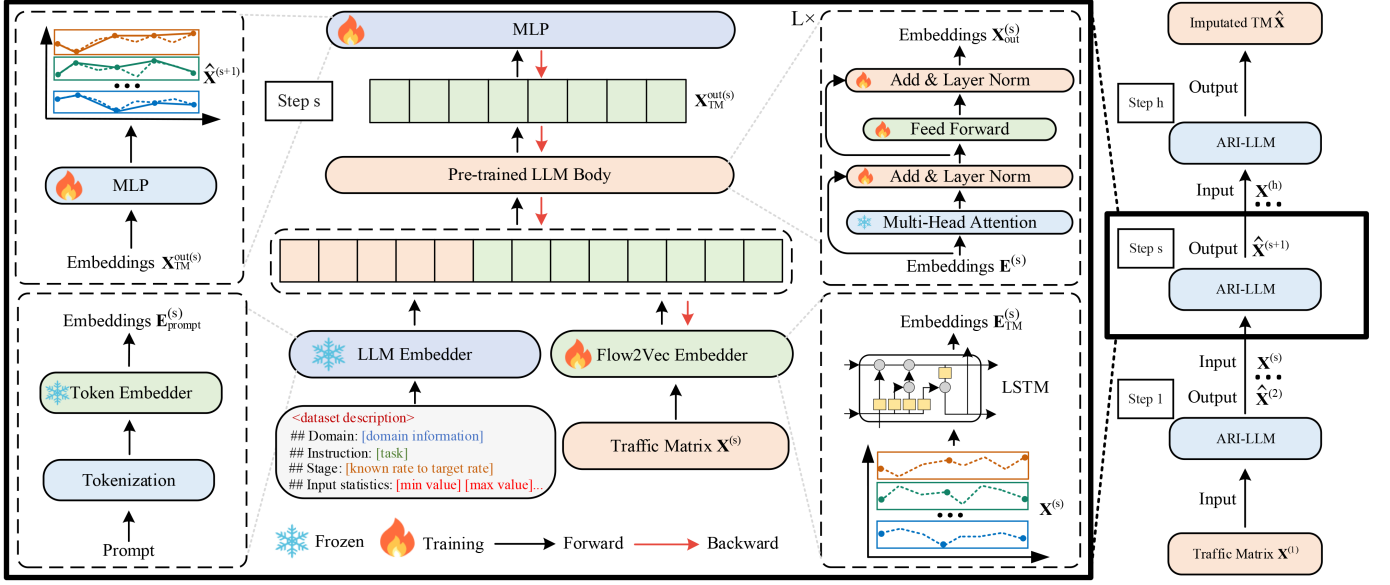


Fig. 2. The framework of ARI-LLM: the Multi-Head Attention layers of the pre-trained LLM are frozen to preserve general language modeling capabilities. The LLM Embedder is also frozen, while the Flow2Vec Embedder is fine-tuned to adapt to the traffic data. Additionally, the Normalization layers, Feed Forward layers, and MLP layer are trainable.

the s -th autoregressive step. The corresponding observations are represented as $\tilde{\mathbf{X}}^{(s)} = \{X_i \in \mathbf{X} \mid i \in \mathcal{I}^s\}$.

$\mathcal{I}^{(0)}$ is initialized to include the starting and ending points. We then determine a minimum observation density $q \in (0, 1]$, which controls the starting observation density. Let $K = \lfloor qw \rfloor$ denote the number of observed time points at the first step. Starting from $s = 1$, we uniformly sample K indices across the entire time range to form the observation set $\mathcal{I}^{(1)}$. Then, at each autoregressive step s ($s > 1$), the model predicts new values at the midpoints between consecutive observed time steps. Therefore, $\mathcal{I}^{(s)}$ can be uniformly formulated by:

$$\mathcal{I}^{(s)} = \begin{cases} \{1, w\}, & s = 0, \\ \mathcal{I}^{(0)} \cup \left\{ \left\lfloor \frac{iw}{K} \right\rfloor \mid i = 1, \dots, K \right\}, & s = 1, \\ \mathcal{I}^{(s-1)} \cup \left\{ \left\lfloor \frac{t_i + t_{i+1}}{2} \right\rfloor \mid t_i, t_{i+1} \in \mathcal{I}^{(s-1)} \right\}, & s > 1. \end{cases} \quad (1)$$

This progressive scheme doubles the observed points at each step, where the unobserved values are simply filled by linear interpolation between the two nearest observed neighbors. In doing this, we can construct a coarse-to-fine sequence of TMs: $\{\mathbf{X}^{(1)}, \mathbf{X}^{(2)}, \dots, \mathbf{X}^{(h)}\}$, where $\mathbf{X}^{(s)} = \text{LinearInterp}(\tilde{\mathbf{X}}^{(s)})$, $s = \{1, 2, \dots, h\}$.

To guide the model in generating a finer-grained TM $\mathbf{X}^{(s+1)}$ based on the previously imputed result $\mathbf{X}^{(s)}$ from step s , on the one hand, we penalize the discrepancy between the predicted $\hat{\mathbf{X}}^{(s+1)}$ and the ground-truth $\mathbf{X}^{(s+1)}$; on the other hand, we leverage task-specific prompts to instruct LLMs to perform finer-grained reasoning.

Considering the significant variability in traffic magnitudes in TM, we adopt the Symmetric Mean Absolute Percentage Error (SMAPE) [34] as the loss function to balance differences across TMs of varying resolutions. Specifically, for the pre-

dicted matrix $\hat{\mathbf{X}}^{(s+1)}$ at step s , and the corresponding ground-truth matrix $\mathbf{X}^{(s+1)}$, the SMAPE loss is defined as:

$$\mathcal{L}_{\text{SMAPE}}^{(s+1)} = \frac{1}{w} \sum_{i=1}^w \frac{\|\hat{\mathbf{X}}_i^{(s+1)} - \mathbf{X}_i^{(s+1)}\|_1}{(\|\hat{\mathbf{X}}_i^{(s+1)}\|_1 + \|\mathbf{X}_i^{(s+1)}\|_1)/2 + \epsilon}, \quad (2)$$

where $\|\cdot\|_1$ denotes the L1-Norm, and ϵ is a small constant added for numerical stability to avoid division by zero.

To enable the LLMs to comprehend the reasoning process from coarse to fine, we design the prompt template as illustrated in the *Instruction* and *Stage* part in Fig. 4. Specifically, *Instruction* guides the model to impute the TM progressively over h stages. The *Stage* specifies the observation ratio at each stage and indicates the target resolution of the next stage.

D. Flow2Vec Embedder

Existing LLM-based models for time series tasks typically treat the full set of observations at each time step as a single unit [18] [35] [36], which is then embedded into the token space of the LLMs. While these approach facilitate learning temporal dependencies across different time series, it inherently requires the input layer to be explicitly tailored to a fixed input dimension. Consequently, when the number of features varies, the input layer must be modified accordingly and the model must be retrained.

To address this limitation and accommodate the dynamic nature of network environment, we propose a novel embedding module, Flow2Vec, which treats each individual traffic flow as an unit. For each flow, we fix a temporal window of length w , treating it as the input dimension. Each flow is represented as a $w \times 1$ vector, which is then embedded into a $d \times 1$ token with the same dimensionality as the word embeddings used in LLMs. Thus, each flow corresponds to a single token, enabling the model to predict the next set of tokens by capturing correlations among the current tokens via self-attention. This

design ensures that the model remains structurally consistent regardless of changes in the number of flows within the TM, just as LLMs can handle sentences of varying lengths. To ensure that the model can still learn the temporal dependencies within each flow, the Flow2Vec embedder incorporates an LSTM encoder to extract temporal features from each flow over the fixed-length window.

The embedding process of Flow2Vec is illustrated in Fig. 3, at the s -th step of the autoregressive process, the current TM $\mathbf{X}^{(s)} \in \mathbb{R}^{w \times n}$ is first transposed and then fed into an LSTM encoder to extract temporal dependencies. It is subsequently embedded into flow-level token embeddings $\mathbf{E}_{\text{TM}}^{(s)} \in \mathbb{R}^{n \times d}$, which are finally input to the LLM for self-attention learning to capture spatial correlations among flows.

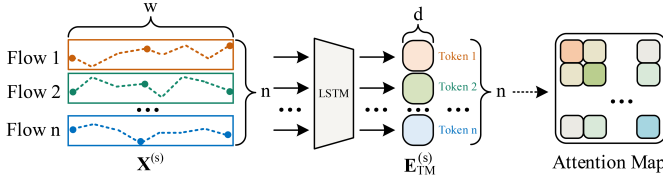


Fig. 3. TM embedding process through Flow2Vec.

Although incorporating more flows in each input allows the LLMs to extract richer inter-flow information, we provide the following theorem to show that increasing the number of flows beyond a certain threshold yields diminishing returns.

Theorem 1 (Optimal Number of Input Flows for LLMs). Assume a TM $\mathbf{X} \in \mathbb{R}^{w \times n}$ has rank r , where n is the total number of flows and w is the time length. Consider a Transformer-based LLM architecture, where each flow is embedded as a token vector and fed into the model. Then, the optimal number of input flows n^* to the LLM that maximizes the imputation performance is $n^* = r$.

Proof. We prove the theorem by analyzing the trade-off between information completeness and model complexity in the context of TM imputation.

Since the TM $\mathbf{X}$ is rank- r , all the intrinsic spatial-temporal information of the matrix lies in an r -dimensional subspace. Thus, observing fewer than r flows ($n < r$) implies incomplete coverage of this subspace, resulting in a biased and underdetermined model that cannot fully reconstruct the TM. Consequently, increasing the number of input flows n for $n \leq r$ leads to a strict decrease in reconstruction error:

$$\frac{d\mathcal{E}_{\text{rec}}(n)}{dn} < 0, \quad \text{for } n \leq r,$$

where $\mathcal{E}_{\text{rec}}(n)$ denotes the prediction error as a function of input flow count.

When $n > r$, additional flows provide redundant or nearly linearly dependent information. While the intrinsic information content saturates at r flows, increasing n further introduces several negative effects: (1) Computational overhead: transformer self-attention complexity scales quadratically with input sequence length, increasing training and inference costs; (2) Model overfitting and noise sensitivity: excessive input tokens increase model capacity requirements and raise variance, which harms generalization under limited training data.

Hence, for $n > r$, the prediction error tends to increase with n :

$$\frac{d\mathcal{E}_{\text{rec}}(n)}{dn} > 0, \quad \text{for } n > r.$$

Combining the above two results, $\mathcal{E}_{\text{rec}}(n)$ is a unimodal function with a minimum at $n = r$. $\square$

According to Theorem 1, the optimal number of flows can be determined by the degree of spatiotemporal correlation of the underlying network's TMs. The stronger the spatiotemporal correlation the smaller of the rank r , and consequently, fewer flows need to be fed into the LLMs in each batch; otherwise, more flows are required. In Section IV.B, we empirically verified the optimal number of flows for each dataset.

Remark 1. Ideally, each batch of inputs to the LLM should consist of r independent flows. However, in real-world networks, the true value of r is often unknown and can only be empirically estimated. Moreover, due to missing data, it is difficult to identify truly independent flows. In our experiments, we randomly divide the flows into $\lceil T/r \rceil$ groups for batch-wise input, which has already achieved satisfactory performance.

E. Prompt Template

To leverage the pretrained knowledge of LLMs for the TM imputation task, we design a prompt template as illustrated in Fig. 4. The starting observation ratio q is set to 2%.

[Database]:	The Abilene dataset contains 12 routers from the U.S. Internet2 Network, 30 directed inner links, and 24 outside links. The dataset collected the volumes of all OD flows in the network every 5 minutes from March to September 2004. Below is the information about the input time series:
[Domain]:	The dataset consists of 12 routers (v1-v12) with 144 OD flows (12×12). Each flow x_{ij} represents traffic from v_i to v_j , indexed as $(i-1) \times 12 + j$ ($i, j \in 1..12$). Temporal sequence length: [seq len] time points. The current traffic matrix has [known_rate] known rate with $1 - [\text{known_rate}]$ missing data. Missing values are initially imputed using two observed neighbors linear interpolation along columns. During the training process, we select fixed-interval rows as sampled data.
[Instruction]:	Learn spatiotemporal patterns in traffic flows, perform progressive matrix imputation through six refinement stages:
[Stage]:	2% $\rightarrow$ 4%, 4% $\rightarrow$ 8%, 8% $\rightarrow$ 16%, 16% $\rightarrow$ 32%, 32% $\rightarrow$ 64%, 64% $\rightarrow$ 100% (current observed data $\rightarrow$ target filling). Current stage is [known_rate] $\rightarrow$ [target_rate].
[Input statistics]:	Min value is [min value]. Max value is [max value]. Median value [median value].

Fig. 4. Example of a prompt template for TMs imputation in the Abilene dataset with starting ratio of 2%: $\square$ represents task-specific configurations and data statistics. $\blacksquare$ indicates the start of a specific type of prompt.

The prompt structure contains a *Head* section describing dataset fundamentals (including name, network topology, and sampling methodology) and a *Load* section with four key components: (a) *Domain* specifies the TM dimensions and data properties; (b) *Instruction* details our autoregressive imputation strategy and task requirements; (c) *Stage* indicates the current phase (e.g., 2% observed data) and target phase (e.g., 4% observed data) of the refinement process; (d) *Statistics* provide key metrics (minimum, maximum, and median values) of the TM. This crafted prompt template allows the LLMs

to properly understand both our coarse-to-fine imputation mechanism and the inherent traffic patterns within TMs.

For a given TM $\mathbf{X}$, we first generate a prompt based on the prompt template. This prompt is then embedded into the token embeddings using the LLM's native tokenization module, resulting in a prompt embedding $\mathbf{E}_{\text{prompt}}$ that shares the same dimensionality as the TM embedding processed by Flow2Vec. The two embeddings are then concatenated and fed into the LLM body for further processing.

F. LLM Body

The input to the LLM body is constructed by concatenating the TM embedding and the prompt embedding:

$$\mathbf{E} = [\mathbf{E}_{\text{TM}}, \mathbf{E}_{\text{prompt}}] \in \mathbb{R}^{(n+m) \times d}, \quad (3)$$

where $\mathbf{E}_{\text{TM}} \in \mathbb{R}^{n \times d}$ is the embedding of n traffic flows and $\mathbf{E}_{\text{prompt}} \in \mathbb{R}^{m \times d}$ is the prompt embedding, where m denotes the number of prompt tokens, and both embeddings share the same token dimension d .

This concatenated input $\mathbf{E}$ is then sequentially passed through L layers of the Transformer-based LLM. Each layer consists of a Multi-Head Attention module (denoted by $\text{Att}()$), a residual connection followed by a layer Normalization (denoted by $\text{Norm}()$), and a Feed Forward network (denoted by $\text{FF}()$). Specifically, for each Transformer layer l , the transformation is given by:

$$\mathbf{H}^{(l)} = \text{FF} \left(\text{Norm} \left(\mathbf{H}^{(l-1)} + \text{Att} \left(\mathbf{H}^{(l-1)} \right) \right) \right), \quad (4)$$

where $\mathbf{H}^{(0)} = \mathbf{E}$, and $\mathbf{H}^{(l)} \in \mathbb{R}^{(n+m) \times d}$ denotes the output of the l -th layer. After passing through all L layers, we obtain the final hidden representation: $\mathbf{X}_{\text{out}} = \mathbf{H}^{(L)}$.

To isolate the model's prediction for the traffic flows, we discard the first m rows of $\mathbf{X}_{\text{out}}$ corresponding to the prompt tokens. The remaining n rows form the TM-related output: $\mathbf{X}_{\text{TM}}^{\text{out}} = \mathbf{X}_{\text{out}}[m+1:]$, which is then fed into a multilayer perceptron (denoted by $\text{MLP}()$) to produce the final prediction of TM:

$$\hat{\mathbf{X}} = \text{MLP}(\mathbf{X}_{\text{TM}}^{\text{out}}). \quad (5)$$

G. Lightweight Fine-Tuning

To maintain the model's pre-trained knowledge while adapting it to the imputation task, we employ a lightweight fine-tuning approach. Specifically, while keeping the Multi-Head Attention layers frozen to preserve their learned representations, we selectively update only four critical components: the Flow2Vec weights (Θ_{f2v}) to tokenize the TM data, the Normalization weights (Θ_{norm}) to maintain stable feature scale, the Feed Forward weights (Θ_{ff}) to facilitate non-linear feature transformations, and the MLP weights (Θ_{mlp}) to map the outputs onto the TM space.

The detailed fine-tuning algorithm is presented in Alg. 1. It inputs a set of training TMs $\mathbf{X}$, an initial observation ratio q , and a learning rate η . It outputs the trained parameters Θ_{llm} , that comprises Θ_{f2v} , Θ_{norm} , Θ_{ff} , and Θ_{mlp} .

The algorithm first initializes the model parameters and the observation index (line 1). Then it calculates the initial number

of observations K and the total number of resolutions h (line 2~3), and performs the autoregressive imputation through h loops (line 4~15). In each loop, it first determines the observed indices $\mathcal{I}^{(s)}$ and $\mathcal{I}^{(s+1)}$ based on Eqn. (1) (line 5). Linear interpolation is then applied to the observed traffic flows to generate the TM in current and next resolutions $\mathbf{X}^{(s)}$ and $\mathbf{X}^{(s+1)}$, respectively (line 6). It transposes $\mathbf{X}^{(s)}$ and feeds it into the Flow2Vec to generate the flow-level embedding $\mathbf{E}_{\text{TM}}^{(s)}$ (line 7). Then, the prompt corresponding to $\mathbf{X}^{(s)}$ is processed by the LLM Embedder to generate the prompt embedding $\mathbf{E}_{\text{prompt}}^{(s)}$ (line 8). The two embeddings are then concatenated to form the embedding $\mathbf{E}^{(s)}$ as the input to the LLM (line 9). After passing $\mathbf{E}^{(s)}$ through the LLM, it obtains the output of the last hidden layer, denoted as $\mathbf{X}_{\text{out}}^{(s)}$ (line 10). The first m prompt tokens is cut off to obtain an output for the TM (line 11), which is then passed through an MLP to produce the final prediction $\hat{\mathbf{X}}^{(s+1)}$ (line 12). The loss between the imputed matrix $\hat{\mathbf{X}}^{(s+1)}$ and the ground truth $\mathbf{X}^{(s+1)}$ is computed in terms of SMAPE (line 13) and added to the total loss $\mathcal{L}$ (line 14). When the iterations terminate, it backpropagates the total loss $\mathcal{L}$ (line 16), and applies gradient descent to update all trainable parameters (line 17). Note that in the final iteration, the generated $\hat{\mathbf{X}}^{(h+1)}$ corresponds to the fully recovered TM $\hat{\mathbf{X}}$, while $\mathbf{X}^{(h+1)}$ is the ground-truth TM $\mathbf{X}$. The associated index set $I^{(h+1)} = \{1, 2, \dots, w\}$ includes all time points.

Algorithm 1 Fine-Tuning Algorithm of ARI-LLM

Input: $\mathbf{X} = \{X_1, X_2, \dots, X_w\}^\top$, q , η
Output: Trained parameters $\Theta_{\text{llm}} = \{\Theta_{\text{f2v}}, \Theta_{\text{norm}}, \Theta_{\text{ff}}, \Theta_{\text{mlp}}\}$

- 1: **Initialize:** $\Theta_{\text{llm}}, \mathcal{I}^{(0)} \leftarrow \{1, w\}$;
- 2: $K = \lfloor qw \rfloor$;
- 3: $h = \lceil \log_2 w/K \rceil$;
- 4: **for** $s = 1$ to h **do**
- 5: Compute $\mathcal{I}^{(s)}$ and $\mathcal{I}^{(s+1)}$ by Eqn. (1);
- 6: $\mathbf{X}^{(s)} \leftarrow \text{LinearInterp}(\{X_i \in \mathbf{X} \mid i \in \mathcal{I}^{(s)}\})$,
 $\mathbf{X}^{(s+1)} \leftarrow \text{LinearInterp}(\{X_i \in \mathbf{X} \mid i \in \mathcal{I}^{(s+1)}\})$;
- 7: $\mathbf{E}_{\text{TM}}^{(s)} \leftarrow \text{Flow2Vec}((\mathbf{X}^{(s)})^\top, \Theta_{\text{f2v}})$;
- 8: $\mathbf{E}_{\text{prompt}}^{(s)} \leftarrow \text{LLMEmbedder}(\text{Prompt}(\mathbf{X}^{(s)}))$;
- 9: $\mathbf{E}^{(s)} \leftarrow [\mathbf{E}_{\text{TM}}^{(s)}, \mathbf{E}_{\text{prompt}}^{(s)}]$;
- 10: $\mathbf{X}_{\text{out}}^{(s)} \leftarrow \text{LLM}(\mathbf{E}^{(s)}; \Theta_{\text{norm}}, \Theta_{\text{ff}})$;
- 11: $\mathbf{X}_{\text{TM}}^{\text{out}(s)} \leftarrow \mathbf{X}_{\text{out}}^{(s)}[m+1:]$;
- 12: $\hat{\mathbf{X}}^{(s+1)} \leftarrow \text{MLP}(\mathbf{X}_{\text{TM}}^{\text{out}(s)}; \Theta_{\text{mlp}})$;
- 13: $\mathcal{L}_s \leftarrow \text{SMAPE}(\hat{\mathbf{X}}^{(s+1)}, \mathbf{X}^{(s+1)})$;
- 14: $\mathcal{L} \leftarrow \mathcal{L} + \mathcal{L}_s$;
- 15: **end for**
- 16: $\nabla_{\Theta_{\text{llm}}} \mathcal{L} \leftarrow \text{Backpropagation}(\mathcal{L})$;
- 17: $\Theta_{\text{llm}} \leftarrow \Theta_{\text{llm}} - \eta \nabla_{\Theta_{\text{llm}}} \mathcal{L}$
- 18: **return** Θ_{llm}

IV. EXPERIMENTS

In this section, we conduct comprehensive experiments to evaluate the performance of ARI-LLM. We compare our method with seven state-of-the-art baselines across three benchmark datasets.

A. Experimental Setup

1) *Datasets*: We use three real-world datasets from different network environments. The Abilene dataset [9], contains OD flows' measurements between 12 network nodes, recorded every 5 minutes over a period of 168 days. The GÉANT dataset [37], contains the flows between 23 nodes, taken every 15 minutes for 112 consecutive days. The WS-DREAM dataset [38] focuses on throughput measurements between 4,500 web services and 142 users, collected at 15-minute intervals during 64 continuous time slots. We split each dataset into training, validation, and testing sets in an 8:1:1 ratio.

2) *Evaluation Metrics*: Following most imputation methods, we employ three metrics: Normalized Mean Absolute Error (NMAE), Normalized Root Mean Square Error (NRMSE) and Kullback-Leibler Divergence (KL) to evaluate the performance of imputation methods. NMAE and NRMSE evaluate the errors of the predicted values compared to the ground truth. KL divergence measures the similarity between the distributions of the predicted and real TM data, where a smaller KL value implies a closer match.

Table I presents the formulas for each metric. Here, x_{ij} and $\hat{x}_{ij}$ denote the real and predicted values of the j -th OD flow at time point i in $\mathbf{X}$ and $\hat{\mathbf{X}}$, respectively. $p_{ij} = \frac{x_{ij}}{\sum_{(i,j) \in \Phi} x_{ij}}$, $\hat{p}_{ij} = \frac{\hat{x}_{ij}}{\sum_{(i,j) \in \Phi} \hat{x}_{ij}}$ represent the probability density function of the real and predicted flows, respectively. Φ and $\hat{\Phi}$ represent the index sets of observed and unobserved data, respectively. These metrics are computed solely on unobserved data.

TABLE I
PERFORMANCE METRICS

Method	Formula
NMAE	$\frac{\sum_{(i,j) \in \Phi} x_{ij} - \hat{x}_{ij} }{\sum_{(i,j) \in \Phi} x_{ij}}$
NRMSE	$\frac{\sqrt{\sum_{(i,j) \in \Phi} (x_{ij} - \hat{x}_{ij})^2}}{\sqrt{\sum_{(i,j) \in \Phi} x_{ij}^2}}$
KL	$\sum_{(i,j) \in \Phi} p_{ij} \log \left(\frac{p_{ij}}{\hat{p}_{ij}} \right)$

3) *Baselines*: We compare TM-LLM with 7 representative imputation methods, which can be categorized into three groups: (1) deep network embedding methods including CoSTCo [39], NTC [14], NTM [40], and PetTC [13], (2) temporal network embedding methods, including NTF [41] and LTP [31], and (3) GAN-based method DATC [17]. Among them, NTC, DATC, LTP and PetTC are specifically designed for network TM imputation, while NTM, NTF, CoSTCo are widely applied in recommendation systems.

4) *Experimental settings*: All experiments were conducted on a single NVIDIA Tesla A800 80G GPU. We employed GPT-2 as our base pre-trained model. The initial observation rate $q = 0.02$. We set the window size $w = 50$ for each input TM. The results presented below represent the average of five runs, adhering to the experimental settings outlined in [13]. Further detailed settings of our experiments are available at <https://anonymous.4open.science/t/ARI-LLM-E7A3>.

B. Optimal Flow Number for Each Dataset

As mentioned in Section III.D, there exists an optimal number of flows for best performance. We validate this empirically

in this subsection under the observation rate of 10%.

Fig. 5 illustrates the imputation errors in terms of NMAE as well as the training time under different numbers of input flows. According to Fig. 5, as the number of input flows increases, the imputation error initially declines, and then either stabilizes or increases slightly. For the Abilene dataset, the number of input flows required to achieve the lowest imputation error is nearly equal to the total number of flows in the network. This indicates the spatiotemporal correlations among flows are not particularly strong. In contrast, for the GÉANT dataset, the imputation error stabilizes when the number of input flows reaches approximately 300, indicating relatively stronger spatiotemporal correlations among flows in the GÉANT network. For WS-DREAM, the lowest error occurs when the number of input flows is 550. Moreover, as the number of input flows increases, the model becomes more difficult to train, resulting in significantly longer training times. Based on this analysis, we set the number of input flows to 144 for Abilene, 300 for GÉANT, and 550 for WS-DREAM in the subsequent experiments.

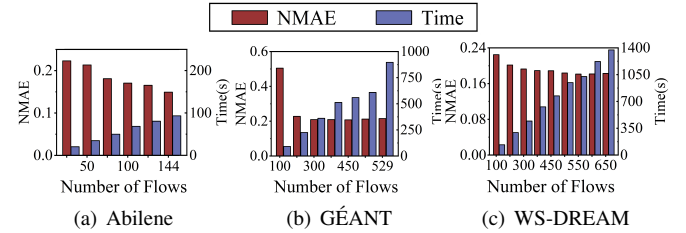


Fig. 5. The optimal number of input flows for the three datasets.

C. Imputation Performance under Various Sampling Rates

Table II summarizes the performance of the eight TM imputation methods on three datasets under extremely low observation rates (OR) from 2% to 10%. From the table, ARI-LLM achieves the best performance on nearly all metrics, consistently outperforming the other seven baselines.

Specifically, compared to the best-performing baseline PetTC, ARI-LLM improves the NMAE by 11.8% ~ 18.1% on Abilene, 0.8% ~ 8.6% on GÉANT, and 27.9% ~ 34.7% on WS-DREAM. For the NRMSE metric, the gains are 3.7% ~ 11.8% on Abilene, 2.3% ~ 13.9% on GÉANT, and 14.3% ~ 22.8% on WS-DREAM, respectively. The accuracy gains are especially pronounced on WS-DREAM, which involves high-dimensional traffic data, indicating the model's strong robustness in large-scale networks.

From different observation ratios, ARI-LLM averagely reduces the best-performing baseline's errors by 13.2%, 14.6%, 16.2%, 16.5%, and 19.6% under the observation rates from 10% to 2%, respectively. The performance gap widens as the observation rate decreases, demonstrating our model's robustness under sparse imputation scenarios.

Regarding KL divergence, ARI-LLM achieves average improvements of 25.6% on Abilene, 37.3% on GÉANT, and 30.8% on WS-DREAM. This demonstrates ARI-LLM can generate TMs that are significantly closer to the true distribution. The above results highlight the powerful TM imputation and pattern learning capability from the synergy between autoregressive imputation and the pretrained LLM.

TABLE II
IMPUTATION PERFORMANCE ON ABILENE, GÉANT AND WS-DREAM. **BOLD**: BEST, UNDERLINED: SECOND BEST.

Dataset		Abilene					GÉANT					WS-DREAM				
Metric	Model	OR 2%	OR 4%	OR 6%	OR 8%	OR 10%	OR 2%	OR 4%	OR 6%	OR 8%	OR 10%	OR 2%	OR 4%	OR 6%	OR 8%	OR 10%
NMAE	ARI-LLM	0.1839	0.1638	0.1568	0.1543	0.1492	0.2666	<u>0.2349</u>	0.2242	0.2189	<u>0.2076</u>	0.2376	0.1940	0.1852	0.1833	0.1812
	PetTC	0.2247	0.1925	0.1843	0.1764	0.1692	0.2917	0.2345	0.2264	0.2208	0.2014	0.3513	0.2974	0.2762	0.2647	0.2515
	LTP	0.2585	0.2293	0.2257	0.2144	0.2031	0.3694	0.3286	0.3008	0.2858	0.2788	0.4087	0.3855	0.3525	0.3413	0.3405
	DATC	0.2954	0.2748	0.2631	0.2596	0.2541	0.4939	0.4332	0.3651	0.3215	0.3037	0.9210	0.8431	0.8536	0.7734	0.8125
	NTC	0.3702	0.3362	0.3243	0.3115	0.3073	0.7042	0.6796	0.6239	0.5781	0.4751	0.4931	0.4983	0.4818	0.4619	0.4355
	NTM	1.2214	0.8818	0.7633	0.6686	0.6178	1.4547	1.3646	1.3266	1.2003	1.1978	0.6337	0.7210	0.6502	0.6390	0.6320
	NTF	0.7529	0.4147	0.3637	0.3486	0.3353	0.9061	0.7783	0.7189	0.5958	0.5125	0.6788	0.5335	0.4976	0.4620	0.4114
	CoSTCo	0.3857	0.3739	0.3381	0.3224	0.3145	0.8588	0.7826	0.7291	0.7241	0.7042	0.5412	0.5321	0.4655	0.4501	0.4430
NRMSE	ARI-LLM	0.2316	0.2123	0.1966	0.1905	0.1818	0.2767	<u>0.2360</u>	0.2221	0.2206	0.2077	0.3525	0.2923	0.2830	0.2790	0.2729
	PetTC	0.2413	<u>0.2205</u>	<u>0.2113</u>	<u>0.2087</u>	<u>0.2062</u>	0.3214	0.2321	<u>0.2298</u>	<u>0.2285</u>	<u>0.2128</u>	0.4116	0.3789	<u>0.3632</u>	<u>0.3576</u>	<u>0.3425</u>
	LTP	0.3173	0.2937	0.2921	0.2908	0.2642	0.2954	0.2614	0.2446	0.2381	0.2329	0.4720	0.4375	0.4112	0.4042	0.3947
	DATC	0.3453	0.3312	0.3226	0.3202	0.3178	0.3818	0.3415	0.2979	0.2547	0.2368	0.9276	0.8721	0.7992	0.7786	0.7473
	NTC	0.4388	0.4072	0.3951	0.3858	0.3766	0.6412	0.6266	0.5888	0.5450	0.4310	0.5308	0.4737	0.4490	0.4259	0.4140
	NTM	0.9720	0.7573	0.7234	0.6613	0.6197	0.9230	0.9057	0.8877	0.7854	0.7629	0.6229	0.5613	0.5788	0.5389	0.5128
	NTF	0.6856	0.4201	0.3882	0.3848	0.3723	0.7110	0.6621	0.5838	0.4959	0.4128	0.6206	0.5105	0.4782	0.4757	0.4379
	CoSTCo	0.4081	0.3929	0.3736	0.3666	0.3600	0.7215	0.6579	0.6252	0.6037	0.5927	0.5286	0.4643	0.4595	0.4234	0.4120
KL	ARI-LLM	0.0493	0.0393	0.0355	0.0336	0.0302	0.1397	0.1011	0.0886	0.0843	0.0772	<u>0.1463</u>	0.0820	0.0652	0.0592	0.0587
	PetTC	0.0532	<u>0.0513</u>	<u>0.0501</u>	<u>0.0498</u>	<u>0.0471</u>	0.2593	0.1408	0.1308	0.1419	0.1280	0.1237	0.1145	<u>0.1076</u>	<u>0.1053</u>	<u>0.1029</u>
	LTP	0.5021	0.1832	0.1524	0.1336	0.1004	0.7012	0.4531	0.4254	0.3755	0.3491	0.4641	0.3573	0.3214	0.2561	0.2139
	DATC	0.0994	0.0881	0.0794	0.0767	0.0736	0.4495	0.3338	0.2522	0.2327	0.2103	1.5789	1.2631	1.0789	0.7368	0.7894
	NTC	0.1578	0.1326	0.1202	0.1157	0.1095	1.2776	1.0643	0.8378	0.8306	0.4765	0.4814	0.3476	0.3015	0.3289	0.1952
	NTM	2.5215	0.9318	0.8276	0.7838	0.6946	2.0707	1.9729	1.8230	1.7226	1.5882	2.3166	0.2011	0.1974	0.1553	0.1102
	NTF	0.4744	0.2114	0.1635	0.1591	0.1228	1.1552	0.7186	0.6736	0.6624	0.5734	0.2431	0.1572	0.1084	0.1057	0.1041
	CoSTCo	0.3566	0.4370	0.3305	0.2851	0.2461	5.2273	3.7190	3.4680	3.3872	3.0307	0.2995	0.2439	0.1911	0.1768	0.1244

D. Ablation Study

To verify the effectiveness of the key designs in ARI-LLM, we conduct an ablation study by implementing two variants of the model: ARI-LLM w/o Prompt and ARI-LLM w/o AR. ARI-LLM w/o Prompt discards the prompt input, relying solely on TM input without textual guidance. ARI-LLM w/o AR removes the autoregressive learning/imputation mechanism, and applies a single-step imputation strategy.

Fig. 6 presents the results of the ablation study at 10% observation rate. As shown in the figure, the original model significantly outperforms the two ablated variants, validating the effectiveness of both key components. Between them, ARI-LLM w/o AR shows a larger performance drop than ARI-LLM w/o prompt, indicating the critical role of the autoregressive mechanism for LLM-based imputation.

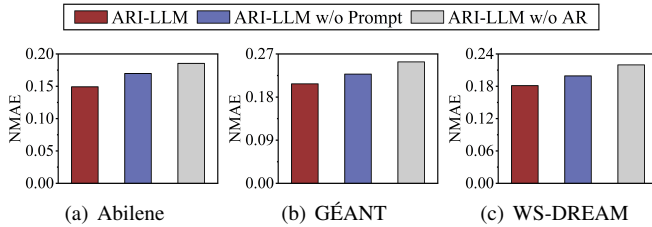


Fig. 6. Results on ablation study under 10% observation rate.

E. Performance in Dynamic Network Environments

To simulate the dynamic network environments, we randomly select $p\%$ of all flows in each dataset for training and then introduce the remaining flows during testing. This setup is designed to assess the model's ability to adapt to

newly emerging traffic patterns when the underlying network structure undergoes varying degrees of changes.

Fig. 7 presents the imputation errors of ARI-LLM and the best-performing baseline, PetTC¹, under varying proportions of flows used for training. As shown in the figure, although both methods exhibit increased errors as the proportion of training flows decreases, the error of PetTC rises significantly faster than that of ARI-LLM. Specifically, when only 20% of flows are used for training, the average error of ARI-LLM across the three datasets increases by just 7.1%, while the error for PetTC rises by 22.3%. This demonstrates the strong generalization ability of ARI-LLM.

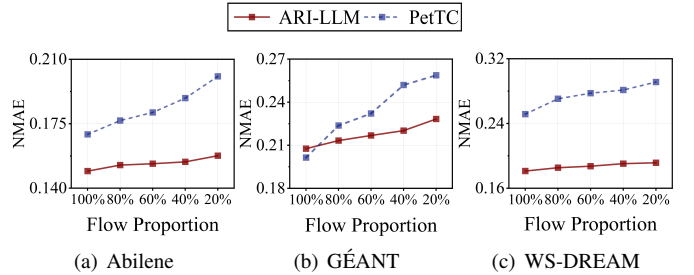


Fig. 7. Imputation errors in dynamic network environments under 10% observation rate.

F. Performance in Unseen Networks

Experiments in this subsection evaluate the applicability of the two methods in a more challenging scenario: unseen

¹To enable PetTC to handle TMs of varying dimensions, we introduce a linear layer to its input to project each TM into a fixed-length representation compatible with the model's input size.

network without any prior knowledge. In such scenario, the model is trained on the TM data from one network and then used to predict the missing flows in a different network.

In Fig. 8(a), A→A denotes both the training and testing flows are from the Abilene network; G→A indicates the model is trained on flows from the GÉANT network and tested on Abilene; W→A means training is done on WS-DREAM and testing on Abilene. In Fig. 8(b) and (c), the corresponding symbols follow the same convention. As shown in the figure, although ARI-LLM has never seen the traffic from the target network, it still achieves relatively stable and reliable imputation performance. In contrast, PetTC exhibits a significant performance drop when applied to unseen networks. Specifically, ARI-LLM experiences an accuracy degradation of 7.6% in Abilene, 8.7% in GÉANT, and 8.3% in WS-DREAM on average, while PetTC suffers from a severe degradation of 23.2% in Abilene, 32.5% in GÉANT, and 23.3% in WS-DREAM. This demonstrates the pre-trained knowledge embedded in the LLM within ARI-LLM endows it with strong zero-shot and knowledge transfer capabilities, significantly enhancing its adaptability to unseen networks.

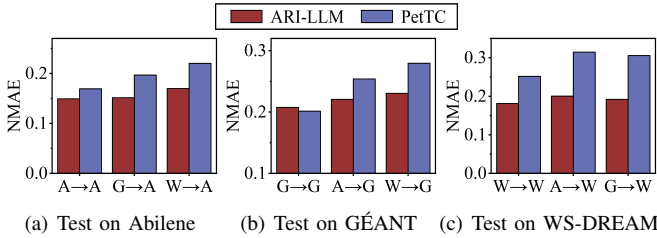


Fig. 8. Imputation errors on unseen networks under 10% observation rate: A: Abilene, G: GÉANT, W: WS-DREAM.

G. Performance with Limited TM snapshots

Considering the high cost of TM data collection, we evaluate the performance of the two methods under limited TM snapshots. Specifically, we train the model using only the first 5% or 10% of the full snapshots, and then apply it to predict the missing TMs in the test set.

Fig.9 presents the imputation errors of the two methods under three training settings: using the full training set (100%), using 10% and 5% of the training data. As shown in the figure, even with a significantly reduced training data, ARI-LLM maintains relatively stable performance, while PetTC exhibits a sharp drop in accuracy. Specifically, averaging across the three datasets, ARI-LLM’s imputation accuracy drops only 0.6% ~ 1.5% with 10% training data and 0.9% ~ 4.7% with 5%. In contrast, PetTC suffers a much steeper decline, with accuracy dropping by 9.5% ~ 14.5% and 12.4% ~ 18.2%, respectively. These results demonstrate the strong few-shot learning capability of ARI-LLM, enabling significant reductions in training costs.

H. Running Speed

Table III compares the training and inferring time of each imputation method. The training time refers to the duration from feeding the training data into the model to its convergence, while the inference time is measured as the total time taken to recover all testing data under a 10% sampling rate.

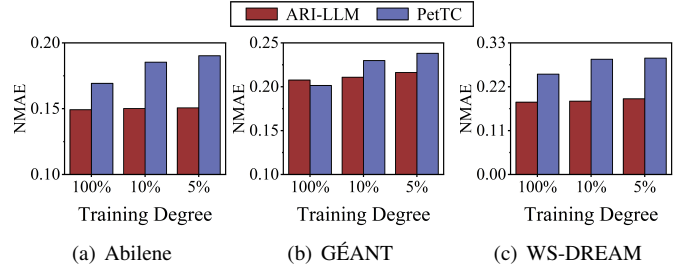


Fig. 9. Imputation errors on limited TM snapshots under 10% observation rate.

As shown in Table III, the training and testing times for all models are shortest on the Abilene dataset and longest on the WS-DREAM dataset. This is because Abilene has the smallest network size and the lowest TM dimension, while WS-DREAM has the largest. Surprisingly, despite ARI-LLM adopting a step-wise autoregressive strategy for training and imputation, its training and inferring times do not increase significantly. On the contrary, ARI-LLM demonstrates superior speed on large-scale networks compared to other baselines.

The key reasons are as follows: (1) The autoregressive scheme in ARI-LLM generates TM blocks of varying resolutions at each step, rather than individual flows, enabling batch processing that improves efficiency. (2) Unlike other models with large numbers of trainable parameters, ARI-LLM involves only lightweight fine-tuning on the LLM. (3) With the coarse-to-fine autoregressive training and prompt-based guidance, the model converges rapidly, typically within just 2~3 training epochs.

TABLE III
TRAINING AND INFERRING TIME IN SECONDS [S]. **BOLD**: BEST, UNDERLINED: SECOND BEST.

Model \ Datasets	Abilene		GÉANT		WS-DREAM	
	Training	Inferring	Training	Inferring	Training	Inferring
NTM	128.0	<u>2.8</u>	1460.5	9.2	5202.0	266.4
NTC	618.1	4.2	3506.3	13.7	2177.1	228.6
CoSTCo	596.2	3.2	<u>526.2</u>	65.1	<u>1487.8</u>	<u>20.6</u>
NTF	84.8	2.9	678.3	10.3	3019.4	207.9
LTP	239.6	2.9	1373.0	10.3	3829.0	209.0
DATC	1728.5	4.5	1876.2	6.1	7548.6	23.2
PetTC	160.3	4.0	1723.1	13.0	2127.8	253.3
ARI-LLM	<u>93.2</u>	1.3	360.7	<u>8.4</u>	1024.8	16.4

V. CONCLUSION

In this work, we introduced ARI-LLM, a novel framework for TM imputation. Inspired by human-like reasoning, our ARI-LLM models the imputation process as a progressive refinement task, gradually enhancing TM quality from coarse to fine. To enable a plug-and-play imputation framework, we developed a specialized embedding scheme and prompt template that enable LLMs to leverage their powerful few-shot and zero-shot capabilities, thereby enhancing generalization to evolving and unseen networks. Extensive experiments on real-world datasets demonstrate ARI-LLM greatly outperforms existing state-of-the-art methods and exhibits outstanding performance when faced with dynamic or unseen networks.

REFERENCES

- [1] Y.-F. Liu, K. C.-J. Lin, and C.-C. Tseng, "Dynamic cluster-based flow management for software defined networks," *IEEE Transactions on Services Computing*, vol. 15, no. 1, pp. 361–371, 2019.
- [2] S. H. Rafique, A. Abdallah, N. S. Musa, and T. Murugan, "Machine learning and deep learning techniques for internet of things network anomaly detection—current research trends," *Sensors*, vol. 24, no. 6, p. 1968, 2024.
- [3] Y. Xie, F. Yi, and K. Jamieson, "Pbe-cc: Congestion control via endpoint-centric, physical-layer bandwidth measurements," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 451–464.
- [4] Í. Cunha, R. Teixeira, D. Veitch, and C. Diot, "Predicting and tracking internet path changes," in *Proceedings of the ACM SIGCOMM 2011 Conference*, 2011, pp. 122–133.
- [5] Q. Zhao, Z. Ge, J. Wang, and J. Xu, "Robust traffic matrix estimation with imperfect information: Making use of multiple data sources," in *Proceedings of the joint international conference on Measurement and modeling of computer systems*, 2006, pp. 133–144.
- [6] Y. Qiao, X. Yuan, and K. Wu, "Routing-oblivious network tomography with flow-based generative model," in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2024, pp. 2139–2148.
- [7] Z. Hu and J. Luo, "Cracking network monitoring in dcns with sdn," in *2015 IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2015, pp. 199–207.
- [8] A. Tootoonchian, M. Ghobadi, and Y. Ganjali, "Opentm: traffic matrix estimator for openflow networks," in *International Conference on Passive and Active Network Measurement*. Springer, 2010, pp. 201–210.
- [9] Y. Zhang, M. Roughan, N. Duffield, and A. Greenberg, "Fast accurate computation of large-scale ip traffic matrices from link loads," *ACM SIGMETRICS Performance Evaluation Review*, vol. 31, no. 1, pp. 206–217, 2003.
- [10] E. D. Biyar, B. Karanlik, and B. Canberk, "Flow-based network tomography agent for software defined data center," in *2019 3rd International Conference on Recent Advances in Signal Processing, Telecommunications & Computing (SigTelCom)*. IEEE, 2019, pp. 170–175.
- [11] A. Medina, N. Taft, K. Salamatian, S. Bhattacharyya, and C. Diot, "Traffic matrix estimation: Existing techniques and new directions," *ACM SIGCOMM Computer Communication Review*, vol. 32, no. 4, pp. 161–174, 2002.
- [12] C. Tebaldi and M. West, "Bayesian inference on network traffic using link count data," *Journal of the American Statistical Association*, vol. 93, no. 442, pp. 557–573, 1998.
- [13] H. Wang, K. Xie, J. Wen, G. Zhang, W. Liang, G. Xie, and K. Li, "Pettc: Pairwise joint embedding based contrastive tensor completion for network traffic monitoring services," *IEEE Trans. Serv. Comput.*, vol. 18, no. 1, pp. 357–371, 2025.
- [14] K. Xie, H. Lu, X. Wang, G. Xie, Y. Ding, D. Xie, J. Wen, and D. Zhang, "Neural tensor completion for accurate network monitoring," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 1688–1697.
- [15] P. Tune and M. Roughan, "Spatiotemporal traffic matrix synthesis," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, 2015, pp. 579–592.
- [16] M. Roughan, Y. Zhang, W. Willinger, and L. Qiu, "Spatio-temporal compressive sensing and internet traffic matrices (extended version)," *IEEE/ACM Transactions on Networking*, vol. 20, no. 3, pp. 662–676, 2011.
- [17] K. Xie, Y. Ouyang, X. Wang, G. Xie, K. Li, W. Liang, J. Cao, and J. Wen, "Deep adversarial tensor completion for accurate network traffic measurement," *IEEE/ACM Transactions on Networking*, vol. 31, no. 5, pp. 2101–2116, 2023.
- [18] M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-F. Li, S. Pan, and Q. Wen, "Time-LLM: Time series forecasting by reprogramming large language models," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [19] H. Wang, J. Qin, A. Bastola, X. Chen, J. Suchanek, Z. Gong, and A. Razi, "Visiongpt: Llm-assisted real-time anomaly detection for safe visual navigation," *arXiv preprint arXiv:2403.12415*, 2024.
- [20] H. Abburi, M. Suesserman, N. Pudota, B. Veeramani, E. Bowen, and S. Bhattacharya, "Generative ai text classification using ensemble llm approaches," *arXiv preprint arXiv:2309.07755*, 2023.
- [21] T. Cui, X. Lin, S. Li, M. Chen, Q. Yin, Q. Li, and K. Xu, "Trafficllm: Enhancing large language models for network traffic analysis with generic traffic representation," *arXiv preprint arXiv:2504.04222*, 2025.
- [22] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "Netllm: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 661–678.
- [23] K. Tian, Y. Jiang, Z. Yuan, B. Peng, and L. Wang, "Visual autoregressive modeling: Scalable image generation via next-scale prediction," *Advances in neural information processing systems (NeurIPS)*, vol. 37, pp. 84 839–84 865, 2024.
- [24] Y. Vardi, "Network tomography: Estimating source-destination traffic intensities from link data," *Journal of the American statistical association*, vol. 91, no. 433, pp. 365–377, 1996.
- [25] Y. Qiao, Z. Hu, and J. Luo, "Efficient traffic matrix estimation for data center networks," in *2013 IFIP Networking Conference*. IEEE, 2013, pp. 1–9.
- [26] J. Cao, D. Davis, S. Vander Wiel, and B. Yu, "Time-varying network tomography: Router link data," *Journal of the American statistical association*, vol. 95, no. 452, pp. 1063–1075, 2000.
- [27] Y. Zhang, M. Roughan, N. Duffield, and A. Greenberg, "Fast accurate computation of large-scale ip traffic matrices from link loads," *ACM SIGMETRICS Performance Evaluation Review*, vol. 31, no. 1, pp. 206–217, 2003.
- [28] K. Xie, X. Wang, X. Wang, Y. Chen, G. Xie, Y. Ouyang, J. Wen, J. Cao, and D. Zhang, "Accurate recovery of missing network measurement data with localized tensor completion," *IEEE/ACM Transactions on Networking*, vol. 27, no. 6, pp. 2222–2235, 2019.
- [29] J. Tian, K. Xie, X. Wang, J. Wen, G. Xie, J. Cao, W. Liang, and K. Li, "Reducing network distance measurement overhead: A tensor completion solution with a new minimum sampling bound," *IEEE Transactions on Networking*, vol. 33, no. 1, pp. 255–270, 2025.
- [30] Y. Qiao, K. Wu, and X. Yuan, "Autotomo: Learning-based traffic estimator incorporating network tomography," *IEEE/ACM Transactions on Networking*, vol. 32, no. 6, pp. 4644–4659, 2024.
- [31] Y. Ouyang, K. Xie, X. Wang, J. Wen, and G. Zhang, "Lightweight trilinear pooling based tensor completion for network traffic monitoring," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 2128–2137.
- [32] C. Liu, K. Xie, T. Wu, C. Ma, and T. Ma, "Distributed neural tensor completion for network monitoring data recovery," *Information Sciences*, vol. 662, p. 120259, 2024.
- [33] Y. Zhou, N. Yu, and Z. Liu, "Towards interactive research agents for internet incident investigation," in *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, 2023, pp. 33–40.
- [34] V. Kreinovich, H. T. Nguyen, and R. Ouncharoen, "How to estimate forecasting quality: A system-motivated derivation of symmetric mean absolute percentage error (smape) and other similar characteristics," Department of Computer Science, University of Texas at El Paso, Technical Report 865, 2014.
- [35] T. Zhou, P. Niu, L. Sun, R. Jin *et al.*, "One fits all: Power general time series analysis by pretrained llm," *Advances in neural information processing systems (NeurIPS)*, vol. 36, pp. 43 322–43 355, 2023.
- [36] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," *Advances in neural information processing systems (NeurIPS)*, vol. 34, pp. 22 419–22 430, 2021.
- [37] S. Uhlig, B. Quoitin, J. Lepropre, and S. Balon, "Providing public intradomain traffic matrices to the research community," *ACM SIGCOMM Computer Communication Review*, vol. 36, no. 1, pp. 83–86, 2006.
- [38] Z. Zheng and M. R. Lyu, "Ws-dream: A distributed reliability assessment mechanism for web services," in *2008 IEEE International Conference on Dependable Systems and Networks With FTCS and DCC (DSN)*. IEEE, 2008, pp. 392–397.
- [39] H. Liu, Y. Li, M. Tsang, and Y. Liu, "Costco: A neural tensor completion model for sparse tensors," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 324–334.
- [40] H. Chen and J. Li, "Neural tensor model for learning multi-aspect factors in recommender systems," in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, 2020, pp. 2449–2455.
- [41] X. Wu, B. Shi, Y. Dong, C. Huang, and N. V. Chawla, "Neural tensor factorization for temporal interaction learning," in *Proceedings of the Twelfth ACM international conference on web search and data mining*, 2019, pp. 537–545.